07_chip_finish 스크립트는 Synopsys **IC Compiler II (ICC2)** 환경에서 레이아웃 설계의 최종 마무리를 짓고, 실리콘 웨이퍼 제조 공장(Foundry) 및 다음 검증 단계로 넘겨줄 핵심 파일들을 추출하는 **Chip Finish (**칩 마무리 및 데이터 출력**)** 단계의 전체 Tcl 스크립트입니다.

배선 최적화까지 끝난 디자인 데이터베이스를 가져와 빈 공간을 메우는 필러 셀(Filler Cell)을 삽입하고, 최종 물리/논리 데이터인 **Verilog Netlist, GDSII, DEF, SPEF** 파일을 일괄 생성하는 테이프 아웃(Tape-out) 직전의 핵심 공정입니다.

---

1. 초기 환경 설정 및 라이브러리 로드

tcl

```
#made by kjinsoo
# module load
#module load icc2/2021.06-SP5
# open icc2_shell
#icc2_shell -gui | tee -i ./09_chip_finish.log

#set current step & before step
set step chip_finish
set before_step route_opt
#source design_setup.tcl
source -e ./icc2_common_env/design_setup.tcl

#set_host_options -max_cores 16

#open lib
open_lib ../06_route/nlib/ORCA_${before_step}.nlib
#copy lib
copy_lib -from ../06_route/nlib/ORCA_${before_step}.nlib -to
./nlib/ORCA_${step}.nlib
#close & open current lib
close_lib -all
open_lib ./nlib/ORCA_${step}.nlib
#open block
open_block ./nlib/ORCA_${step}.nlib:ORCA.design
#link_block
link_block -rebind
```

코드를 사용할 때는 주의가 필요합니다.

- 해설**:** 최종 마무리 단계 실행 로그(`09_chip_finish.log`) 및 단계 변수를 선언합니다. 이전 배선 최적화 단계(`route_opt`)의 결과물 라이브러리를

안전하게 복사하여 현재 `chip_finish` 독립 라이브러리로 오픈 및 동기화(`link_block -rebind`)합니다.

---

2. 스탠다드 셀 필러(Filler Cell) 삽입 및 자동 제어 규칙 적용

tcl

```
####################
#insert_filler
####################
set SH_FILLERS "*/SHFILL128_HVT */SHFILL64_HVT */SHFILL3_HVT
*/SHFILL2_HVT */SHFILL1_HVT"
create_stdcell_fillers -lib_cells $SH_FILLERS -rules
{post_route_auto_delete} ;# it allows route_opt & eco_opt to
automatically remove filler cells
```

코드를 사용할 때는 주의가 필요합니다.

- **`set SH_FILLERS ...`**: 크기별(128, 64, 3, 2, 1 그리드 크기) HVT 스탠다드 셀 필러의 목록을 변수에 지정합니다.
- **`create_stdcell_fillers ... -rules {post_route_auto_delete}`**: 스탠다드 셀 배치 배치 행(Row) 사이에 전력선(Well/Diffusion)이 끊기지 않고 이어지도록 빈 공간을 채우는 필러 셀을 삽입합니다.
- 이때 `-rules {post_route_auto_delete}` 옵션을 사용하여, 추후 엔지니어링 변경 명령(ECO)이나 추가 최적화가 실행될 때 툴이 공간 확보를 위해 이 필러 셀들을 알아서 자동으로 지우고 비켜설 수 있도록 속성을 부여합니다.

---

3. 삽입된 필러 셀의 전원 및 접지(PG Net) 수동 결선

tcl

```
#connect pg_net for all STD filler cell (VDD, VSS)
set FILLERS [get_cells -filter "full_name =~ *SHFILL*"]

set i 0

foreach x $FILLERS {
set pins [get_pins -of_objects $x -filter "port_type == power"]
connect_pg_net -net VDD [get_pins $pins]

incr i
}

foreach x $FILLERS {
```

```
set pins [get_pins -of_objects $x -filter "port_type == ground"]
connect_pg_net -net VSS [get_pins $pins]

incr i
}
```

코드를 사용할 때는 주의가 필요합니다.

- **`set FILLERS ... *SHFILL*`:** 방금 칩 빈 공간에 밀어 넣은 모든 필러 셀 오브젝트를 컬렉션으로 수집합니다.
- **`foreach x $FILLERS { ... }`:** 수집된 필러 셀들을 하나씩 전수 조사 루프를 돌립니다.
- **`connect_pg_net -net VDD / VSS`:** 필러 셀 내부의 전원 타입 핀(`port_type == power`)을 찾아 메인 메탈 전원망인 `VDD` 네트에 물리적으로 묶어주고, 접지 타입 핀(`port_type == ground`)을 찾아 메인 접지망인 `VSS` 네트에 확실하게 연결하여 결선 누락(LVS 에러)을 방지합니다.

---

4. 물리적 위반(DRC)을 유발하는 필러 셀 일괄 제거

tcl

```
#remove Filler Cell with violation
remove_stdcell_fillers_with_violation
```

코드를 사용할 때는 주의가 필요합니다.

- **해설:** 필러 셀이 빽빽하게 삽입되다 보면 공정 미세 규칙(DRC) 경계를 아슬아슬하게 침범하여 쇼트나 이격 거리 위반을 일으키는 개체가 생길 수 있습니다. 이 명령어를 통해 디자인 룰 위반을 유발하는 불량 필러 셀들만 골라내어 안전하게 강제 삭제합니다.

---

5. 게이트 레벨 Verilog Netlist 파일 추출

tcl

```
######################
#write Netlist file
######################
write_verilog ./output/ORCA.v -exclude {leaf_module_declarations
corner_cells pad_spacer_cells filler_cells flip_chip_pad_cells
empty_modules unconnected_ports spare_cells}
```

코드를 사용할 때는 주의가 필요합니다.

- **`write_verilog`:** 테이프 아웃 이후 LVS 검증이나 게이트 레벨 타이밍 시뮬레이션(GLS)에 활용될 최종 구조적 **Verilog** 넷리스트 파일**(`ORCA.v`)**을 출력합니다.
- **`-exclude { ... }`:** 순수한 핵심 로직 게이트 연결 관계만 깔끔하게 보기 위해, 기본 리프 모듈 선언부, 코너 셀, 패드 스페이서, 필러 셀, 빈 모듈 및 연결되지 않은 포트 정보 등은 텍스트 파일에서 제외시켜 용량을 최적화합니다.

---

6. 제조 공정용 최종 물리 레이아웃 GDSII 파일 추출

tcl

```
######################
#write gds
######################
write_gds ./output/ORCA.gds -exclude_empty_block -skip_unplaced_cells
-lib_cell_view layout
```

코드를 사용할 때는 주의가 필요합니다.

- **`write_gds`:** 반도체 파운드리 제조 공장에 넘겨주어 실제 포토마스크를 구울 때 사용하는 가장 중요한 최종 **2D** 바이너리 물리 레이아웃 데이터 포맷 파일**(`ORCA.gds`)**을 생성합니다.
- 내부가 비어있는 블록이나 배치되지 않은 가상 셀 정보는 걸러내고(`-exclude_empty_block`, `-skip_unplaced_cells`), 실제 공정 메탈 마스크 패턴 형태를 온전히 담고 있는 `layout` 뷰를 기준으로 정밀 추출합니다.

---

7. 디자인 교환 포맷 DEF(Design Exchange Format) 파일 추출

tcl

```
######################
#write def
######################
write_def ./output/ORCA.def -include_tech_via_definitions -include
{rows_tracks vias cells ports blockages bounds specialnets nets
routing_rules}
```

코드를 사용할 때는 주의가 필요합니다.

- **`write_def`:** 타사의 EDA 툴(예: Cadence 등)이나 외부 기생 성분 추출 툴과 레이아웃 데이터를 연계 호환할 때 사용하는 디자인 교환 표준 포맷 파일**(`ORCA.def`)**을 추출합니다.
- 비아 정의는 물론 셀 배치 행(Rows), 배선 트랙, 실제 인스턴스 셀 정보, 입출력 포트, 특수 전원망(Specialnets) 및 신호선 배선 가이드라인을 한데 묶어 고스란히 출력합니다.

---

8. RC 기생 성분 추출 파일 SPEF(Standard Parasitic Exchange Format) 생성

tcl

```
######################
#extract_spef (ICC2)
######################
write_parasitics -output ./output/ORCA
```

코드를 사용할 때는 주의가 필요합니다.

- **`write_parasitics`:** 레이아웃에 깔린 모든 메탈 배선과 비아의 기생 저항(R) 및 정전용량(C) 성분을 총망라한 표준 기생 성분 파일**(SPEF)**을 추출합니다. (확장자 `.spef` 등으로 출력됨)
- 이 파일은 다음 공정인 **Sign-off** 타이밍 검증 단계에서 **PrimeTime(PT)** 툴이 실제 정밀 넷 지연**(Net Delay)**을 계산하는 원천 데이터로 활용되는 핵심 파일입니다.

---

9. 마감 저장 및 쉘 종료

tcl

```
#save & exit
save_lib -all
exit
```

코드를 사용할 때는 주의가 필요합니다.

- 해설**:** 필러 셀 삽입 및 마감 처리가 완료된 현재 데이터베이스의 모든 변경 사항을 영구 저장(`save_lib -all`)하고, 정해진 `./output/` 경로에 핵심 결과 파일들을 모두 안전하게 떨구어 낸 뒤 ICC2 자동화 플로우를 최종 종료(`exit`)합니다.

🏁 물리적 설계(P&R) 전체 공정 마무리 요약

축하합니다! 이로써 보내주신 **Power Plan ➔ Placement ➔ CTS ➔ Routing ➔ Chip Finish**에 이르는 디지털 백엔드 물리 설계의 메인 마일스톤 전체 스크립트 리뷰가 완벽하게 완료되었습니다. 이 스크립트가 실행되고 나면 `./output/` 폴더에 반도체 칩 설계의 최종 마스터 피스들이 완성됩니다.